

Observer Pattern in JavaScript

What is the Observer Pattern?

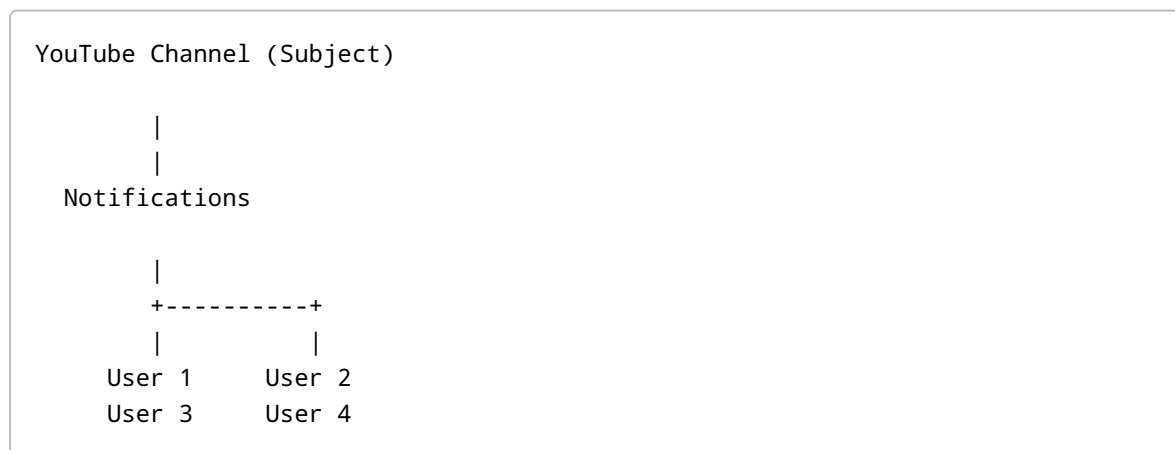
The **Observer Pattern** is a behavioural design pattern where an object (called the **Subject**) maintains a list of dependent objects (called **Observers**) and automatically notifies them when its state changes.

In simple terms:

"When something changes, automatically tell everyone who is interested."

A real-world example:

A YouTube channel:



When the channel uploads a new video, all subscribers are notified.

Why Does the Observer Pattern Exist?

A common problem in software is that multiple parts of an application need to react when something changes.

Example:

An online store:

When an order is completed:

- Send confirmation email.
- Update inventory.
- Notify analytics.
- Update dashboard.

Without Observer:

```
id="withoutObserver"
completeOrder(){

    sendEmail();

    updateInventory();

    updateAnalytics();

    updateDashboard();

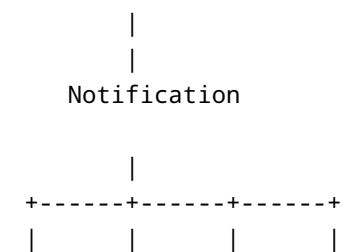
}
```

Problems:

- The order system knows about every service.
- Adding new behaviours requires changing existing code.
- Components become tightly coupled.

With Observer:

Order System (Subject)



Email Inventory Analytics Dashboard

The order system only announces:

"The order changed."

Observers decide what to do.

Core Concepts of Observer Pattern

The Observer Pattern has two main parts.

1. Subject

The object being watched.

Responsibilities:

- Store observers.
- Add observers.
- Remove observers.
- Notify observers.

Example:

```
id="subject"  
NewsPublisher
```

2. Observer

Objects that want updates.

Responsibilities:

- Receive notifications.
- React to changes.

Example:

```
id="observer"  
Subscriber
```

Basic Observer Pattern Example

News Notification System

Imagine a news website.

Users subscribe to updates.

When new news is published, everyone receives a notification.

Creating the Subject

```
id="newsSubject"
class NewsPublisher {

    constructor(){

        // Store all subscribers

        this.subscribers = [];

        this.latestNews = null;

    }

    subscribe(observer){

        // Add a new observer

        this.subscribers.push(observer);

    }

    unsubscribe(observer){

        // Remove an observer

        this.subscribers =
            this.subscribers.filter(
                sub => sub !== observer
            );

    }

    notify(){

        // Tell every observer
        // that something changed

        this.subscribers.forEach(
            subscriber => {

                subscriber.update(
```

```

        this.latestNews
    );
    }
};

}

publish(news){

    this.latestNews = news;

    this.notify();

}

}

```

Creating Observers

Each observer receives updates.

```

id="observerClass"
class Subscriber {

    constructor(name){

        this.name = name;

    }

    update(news){

        console.log(
            `${this.name} received: ${news}`
        );

    }

}

```

Using the Observer Pattern

Create the subject:

```
id="createSubject"  
const news =  
  new NewsPublisher();
```

Create observers:

```
id="createObservers"  
const john =  
  new Subscriber("John");  
  
const mary =  
  new Subscriber("Mary");
```

Subscribe users:

```
id="subscribe"  
news.subscribe(john);  
  
news.subscribe(mary);
```

Publish news:

```
id="publish"  
news.publish(  
  "JavaScript 2026 released!"  
);
```

Output:

```
John received: JavaScript 2026 released!  
  
Mary received: JavaScript 2026 released!
```

How This Works Step-by-Step

1. Create Subject

```
const news = new NewsPublisher();
```

The subject starts with:

```
{  
  subscribers: []  
}
```

2. Add Observers

```
news.subscribe(john);
```

The list becomes:

```
[  
  John  
]
```

Add another:

```
news.subscribe(mary);
```

Now:

```
[  
  John,  
  Mary  
]
```

3. State Changes

Example:

```
news.publish(  
  "New article available"  
);
```

The subject updates:

```
latestNews =  
  "New article available"
```

4. Notify Observers

The subject runs:

```
this.notify();
```

Which calls:

```
observer.update(news);
```

Every subscriber receives the update.

Removing Observers

Observers can unsubscribe.

Example:

```
news.unsubscribe(john);
```

Before:

```
Subscribers:
```

```
John  
Mary
```

After:

Subscribers:

Mary

Now John no longer receives notifications.

A More Realistic JavaScript Example

Stock Price Tracker

Imagine a stock application.

When the price changes:

- Update the graph.
 - Send alerts.
 - Update portfolio value.
-

Subject

```
id="stockSubject"
class Stock {

    constructor(){

        this.observers = [];

        this.price = 0;

    }

    subscribe(observer){

        this.observers.push(observer);

    }

    notify(){
```

```
        this.observers.forEach(
            observer => {

                observer.update(
                    this.price
                );

            }
        );
    }

    setPrice(price){

        this.price = price;

        this.notify();

    }

}
```

Observers

Price Display

```
id="priceDisplay"
class PriceDisplay {

    update(price){

        console.log(
            `Current price: €${price}`
        );

    }

}
```

Alert System

```
id="alertSystem"
class PriceAlert {

    update(price){

        if(price > 100){

            console.log(
                "Price alert triggered!"
            );

        }

    }

}
```

Using It

```
id="stockUsage"
const appleStock =
    new Stock();

appleStock.subscribe(
    new PriceDisplay()
);

appleStock.subscribe(
    new PriceAlert()
);

appleStock.setPrice(120);
```

Output:

Current price: €120

Price alert triggered!

One change triggers multiple reactions.

Observer Pattern Using JavaScript Events

JavaScript already has built-in Observer behaviour.

Examples:

- DOM events.
 - Event listeners.
 - Node.js EventEmitter.
 - Promises.
-

DOM Example

```
id="domObserver"  
button.addEventListener(  
  "click",  
  function(){  
  
    console.log(  
      "Button clicked"  
    );  
  }  
);
```

Here:

Subject:

Button

Observer:

Click handler

When the button changes state:

Clicked

The observer is notified.

Node.js EventEmitter Example

Node.js commonly uses the Observer Pattern.

```
id="nodeObserver"
const EventEmitter =
  require("events");

const emitter =
  new EventEmitter();

emitter.on(
  "message",
  data => {

    console.log(data);

  }
);

emitter.emit(
  "message",
  "Hello Observer"
);
```

Output:

Hello Observer

Common Uses of Observer Pattern in JavaScript

1. Event Handling

Examples:

- Button clicks.
- Keyboard events.
- Mouse events.

Problem solved:

Allow multiple functions to react to the same event.

2. State Management

Examples:

- Redux.
- MobX.
- Vue reactivity.
- React state updates.

Problem solved:

Update components automatically when data changes.

3. Real-Time Applications

Examples:

- Chat applications.
- Stock trackers.
- Notifications.

Problem solved:

Push updates to connected users.

4. Logging Systems

Example:

```
logger.subscribe(  
    monitoringService  
);
```

Observers:

- File logger.
- Monitoring dashboard.
- Alert system.

Problem solved:

Add logging behaviours without changing core code.

5. User Interface Updates

Example:

Changing application state:

```
User Data Changed  
  
    |  
  
+-----+-----+  
  
Profile Cart Dashboard
```

Problem solved:

Keep UI automatically synchronised.

Problems Observer Pattern Solves

Problem 1: Tight Coupling

Without Observer:

```
Order  
  
|  
+--> Email  
  
|  
+--> Inventory
```

```
|  
+--> Analytics
```

Order knows everything.

With Observer:

```
Order  
  
|  
v  
  
Observers  
  
|  
+--> Email  
+--> Inventory  
+--> Analytics
```

Order only sends notifications.

Problem 2: Adding New Behaviours

Without Observer:

Adding analytics requires:

```
id="modifyCode"  
change Order class
```

With Observer:

Add:

```
id="newObserver"  
class AnalyticsObserver {  
  
}
```

No existing code changes.

Problem 3: Keeping Components Synchronized

Example:

A user updates their profile.

Need to update:

- Header.
- Profile page.
- Settings page.

Observer automatically informs all interested components.

Observer vs Strategy Pattern

Observer	Strategy
Notifies multiple objects	Changes one behaviour
Event-driven	Algorithm-driven
Many observers	One selected strategy
"Something happened"	"How should this happen?"

Observer:

```
subject.notify();
```

Strategy:

```
strategy.execute();
```

Observer vs Pub/Sub Pattern

Observer	Pub/Sub
Subject knows observers	Publisher does not know subscribers
Direct relationship	Uses message broker/event bus
Simpler	More decoupled

Observer:

```
Subject
|
+--> Observer
```

Pub/Sub:

```
Publisher
|
Event Bus
|
Subscribers
```

Observer vs Event Listener

They are very similar.

JavaScript event listeners are basically Observer Pattern implementations.

Example:

```
button.addEventListener(  
  "click",  
  handler  
);
```

Button:

```
Subject
```

Handler:

```
Observer
```

Disadvantages of Observer Pattern

1. Memory Leaks

If observers are not removed:

```
id="memoryLeak"  
subject.subscribe(observer);
```

The subject keeps a reference forever.

2. Difficult Debugging

A single event can trigger many observers.

Example:

```
User Update  
  
|  
+--> Email  
+--> Cache  
+--> Analytics  
+--> UI
```

Finding the source can be harder.

3. Unexpected Order

Observers may execute in an order you did not expect.

When Should You Use Observer?

Use Observer when:

- ✓ Multiple objects need updates when something changes
- ✓ You need event-driven communication
- ✓ You want loose coupling
- ✓ New listeners may be added later

Examples:

- UI events.
- Notifications.
- State management.
- Real-time systems.
- Logging.

Avoid Observer when:

- ✗ Only one object needs to react
- ✗ Direct function calls are simpler
- ✗ The notification chain becomes too complex

Key Takeaway

The Observer Pattern separates:

State Change

|

Subject

|

Notifications

|

Observers

Instead of:

Object A

|

+--> Object B

|

+--> Object C

|

+--> Object D

The object causing the change does not need to know what happens afterward.

In JavaScript, the Observer Pattern is commonly implemented using:

1. Custom `subscribe()` / `unsubscribe()` systems.
2. DOM event listeners.
3. Node.js EventEmitter.
4. State management libraries.

It is mainly used to create **loosely coupled systems where objects can react automatically to changes without the main object knowing who is listening.**